

IPST-CNAM Albi | Master 1 TRIED
Module Intelligence Artificielle | Année universitaire 2025-2026
Enseignant : Adrien ESCOURROU

RAPPORT TECHNIQUE

ChatBot IA - Assistant conversationnel Sport, Musculation & Nutrition

Projet guidée - coefficient 3 - Architecture 5 modules - Python 3.12

Auteur : Emmanuel SENAKPON

Date de rendu : 21 aout 2026

Depot Git : https://github.com/emmanuelzenakpon12/chatbot_IA

Table des matières

1. Contexte et besoin.....	5
1.1 Présentation du projet	5
1.2 Domaine choisi : Sport, musculation et nutrition	5
1.3 Public cible.....	6
1.4 Objectifs et contraintes techniques	6
2. Architecture générale du système	7
2.1 Structure du projet.....	7
2.2 Pipeline de traitement d'une question	7
2.3 Diagramme UML des classes	8
2.4 Décisions architecturales majeures	9
2.4.1 Double flux de tokens	9
2.4.2 Recherche par entité séparée.....	9
2.4.3 Persistance du feedback par jeu	9
3. Module 1 - Base de connaissances (knowledge_base.py)	10
3.1 Conception du graphe de connaissances.....	10
3.2 Statistiques et contenu du graphe.....	10
3.3 Implementation des methodes.....	12
3.3.1 add_concept() : idempotence et normalisation	12
3.3.2 add_relation() : validation et bornage	12
3.3.3 get_neighbors() : encapsulation par copie	13
3.3.4 qa_for_concepts() : intersection d'ensembles	13
3.4 Serialisation JSON	13
3.5 Tests unitaires (11 tests).....	13
4. Module 2 - Prétraitement NLP (nlp_engine.py)	14
4.1 Conception générale et choix techniques.....	14
4.2 Stemmer maison SimpleFrenchStemmer.....	14
4.3 Pipeline de prétraitement pas à pas	15
4.4 Classification d'intention (V1 : regles de mots-cles)	16
4.5 Distance de Levenshtein implementee manuellement.....	17
4.6 Extraction d'entites (NER) en 4 strategies en cascade	17
4.7 Tests unitaires NLP (18 tests).....	18

5. Module 3 - Algorithmes de recherche (search_engine.py).....	20
5.1 Modele de cout sur les aretes	20
5.2 BFS - Parcours en largeur	20
5.3 DFS - Parcours en profondeur	21
5.4 A* - Recherche informée et heuristique admissible	21
5.4.1 Principe et implémentation	21
5.4.2 Justification formelle de l'admissibilité	22
5.5 Comparaison mesurée sur 10 requêtes.....	22
5.6 Analyse comparative.....	23
5.7 Orchestration: top_candidates et find_best_answer.....	23
5.8 Tests unitaires (16 tests).....	24
6. Module 4 - Apprentissage (learning_engine.py)	25
6.1 Architecture et philosophie du module	25
6.2 TF-IDF implémentée manuellement.....	25
6.2.1 Pourquoi TF-IDF ?.....	25
6.2.2 Formules implementees	25
6.2.3 Cohérence du stemming : un point critique	26
6.3 Similarite cosinus entre vecteurs creux.....	27
6.4 Naive Bayes multinomial (V2)	27
6.4.1 Principe et lissage de Laplace	27
6.4.2 Donnees d'entrainement et distribution des classes	28
6.5 Boucle de feedback utilisateur	28
6.5.1 Mecanisme complet	28
6.5.2 Demonstration sur 3 cycles reels	29
6.6 Persistence inter-sessions de l'apprentissage	29
6.7 Tests unitaires (17 tests).....	30
7. Module 5 - Intégration et interfaces (main.py / ui.py).....	31
7.1 Orchestrateur ChatBot (main.py).....	31
7.1.1 Gestion de la relance conversationnelle.....	31
7.1.2 Cas limite : question vide.....	31
7.1.3 Recherche par entite separee.....	31
7.2 Interface CLI (run_cli).....	32

7.3 Interface web Flask (bonus).....	32
7.4 Conversation fluide - 20 questions de test	32
8. Metriques et evaluation.....	35
8.1 Precision de la classification d'intention (20 questions)	35
8.2 Precision des reponses (50 questions).....	35
8.3 Temps de reponse sur 20 questions	36
8.4 Comparaison BFS / DFS / A* (10 requetes).....	37
8.5 Impact mesure du feedback sur les poids du graphe.....	37
8.6 Bilan de la suite de tests.....	38
9. Limites identifiées et pistes d'amélioration.....	39
9.1 Limites du systeme actuel.....	39
9.1.1 Couverture incomplete du graphe	39
9.1.2 Stemmer maison imparfait.....	39
9.1.3 Absence de gestion de la negation	39
9.1.4 Questions multi-intentions	39
9.1.5 Re-ranking TF-IDF limite sur les questions generiques	39
9.2 Pistes d'amelioration pour le M2	40
10. Utilisation de l'IA	41
10.1 Outil utilise.....	41
10.2 Usages details.....	41
10.2.1 Génération du corpus de connaissances.....	41
10.2.2 Debug du pipeline NLP	41
10.2.3 Generation de tests unitaires	42
10.4 Taux d'utilisation estime	42
11. Bibliographie et references	43
11.1 Ouvrages de reference.....	43
11.2 Documentation technique	43
11.3 Cours et materiaux pedagogiques	43
11.4 IA utilisee.....	43

1. Contexte et besoin

1.1 Présentation du projet

Ce rapport documente la conception, l'implémentation et l'évaluation d'un assistant conversationnel intelligent réalisé dans le cadre du projet M1 Intelligence Artificielle de l'IPST-CNAM Albi. Le projet, supervisé par M. Adrien ESCOURROU et noté au coefficient 3 (soit 50 % de la note finale du module), demandait de construire de zéro un chatbot complet intégrant les trois grands piliers de l'IA enseignés en M1 à savoir la recherche dans des graphes, l'apprentissage par modèles probabilistes et feedback, et la reconnaissance du langage naturel.

La contrainte principale était l'interdiction d'utiliser toute bibliothèque de Machine Learning externe (sklearn, TensorFlow, PyTorch) ou tout Framework NLP avancé (SpaCy, NLTK sauf SnowballStemmer). Chaque algorithme devait être implémenté manuellement, ce qui en fait un projet à forte dimension pédagogique, on ne se contente pas d'appeler `model.fit()`, on comprend et on code soi-même le TF-IDF, le Naive Bayes, la distance de Levenshtein, et les algorithmes de parcours de graphe.

Le code final représente environ 3 200 lignes de Python réparties sur six modules sources et six fichiers de tests unitaires, pour un total de 102 tests qui passent tous sans la moindre erreur en moins d'une seconde. L'interface CLI est complètement fonctionnelle, et une interface web Flask a été développée en bonus.

1.2 Domaine choisi : Sport, musculation et nutrition

Le choix du domaine n'est pas anecdotique. Pour que la recherche sémantique dans un graphe soit pertinente, il faut un domaine où les concepts sont nombreux, bien délimités, et fortement interconnectés. Le sport, la musculation et la nutrition cohabitent toutes ces cases.

Le vocabulaire est structuré en hiérarchies claires et en réseaux de relations. La musculation implique des protéines, qui sont un sous-ensemble de la nutrition, qui elle-même conditionne la récupération, qui influe sur l'entraînement et l'entraînement relie la musculation au cardio, lui-même lié à l'endurance et au $VO_2\max$. Ces chaînes de relations sémantiques donnent au graphe une densité qui permet à l'algorithme de recherche de naviguer de proche en proche et de trouver des réponses pertinentes même pour des questions indirectes.

Ensuite, les questions posées par les utilisateurs dans ce domaine couvrent naturellement toutes les intentions du sujet : les définitions ("qu'est-ce que le gainage ?"), les questions pratiques ("combien de protéines par jour ?"), les comparaisons ("quelle différence entre cardio et musculation ?"), sans oublier les salutations et les

demandes de quitter. Cette diversité est indispensable pour tester rigoureusement le module de classification d'intention.

Enfin, ce domaine est particulièrement adapté à l'apprentissage par feedback, un utilisateur sait instantanément si la réponse sur les protéines ou la technique de squat est correcte. Le signal de feedback est donc fiable et exploitable.

1.3 Public cible

Le chatbot s'adresse en premier lieu aux débutants en sport qui cherchent des conseils de base sans parcourir des dizaines d'articles : une question naturelle en français, une réponse directe. Il vise aussi les pratiquants réguliers qui souhaitent des informations techniques sur des exercices spécifiques (technique du squat, nombre de répétitions, alimentation pré- et post-séance). Il peut enfin être utilisé dans un contexte éducatif pour illustrer les bases du sport-santé à un public non-spécialiste.

L'un des avantages d'un chatbot de domaine spécialisé par rapport à un grand modèle de langage généraliste est la garantie que les réponses restent dans leur périmètre de compétence. Quand le chatbot ne connaît pas la réponse, il le dit explicitement plutôt que de générer une réponse plausible mais potentiellement incorrecte.

1.4 Objectifs et contraintes techniques

Il était imposé des cibles quantitatives sur chaque module, structurant le développement en cinq étapes successives :

- Etape 1 - Fondations : construire le graphe de connaissances (minimum 30 concepts, 50 relations) et les paires Q/R (minimum 50 paires). Le système final atteint 69 concepts, 118 relations et 72 paires Q/R.
- Etape 2 - NLP : atteindre au moins 80 % de précision sur la classification d'intention sur 20 questions de test. Le système atteint 100 %.
- Etape 3 - Recherche : implémenter BFS, DFS et A* avec heuristique admissible justifiée. Comparer les algorithmes sur au moins 10 requêtes.
- Etape 4 - Apprentissage : TF-IDF et Naive Bayes implémentés manuellement. Le feedback utilisateur doit modifier les poids du graphe et améliorer la précision de plus de 5 % en 3 cycles.
- Etape 5 - Intégration : conversation fluide sur 20 questions minimum. Interface web Flask en bonus. Temps de réponse inférieur à 2 secondes.

2. Architecture générale du système

2.1 Structure du projet

Le projet respecte scrupuleusement l'arborescence imposée par le sujet. Cinq modules sources, un orchestrateur, une interface, et un dossier de tests par module :

```
chatbot_ia/
main.py          # Classe ChatBot : orchestre les 4 modules (192 lignes)
ui.py            # CLI + interface web Flask bonus (727 lignes)
nlp_engine.py    # NLP : tokenisation, stemming, intentions (311 lignes)
knowledge_base.py # Graphe de connaissances orienté pondéré (121 lignes)
search_engine.py # BFS, DFS, A* avec instrumentation (311 lignes)
learning_engine.py # TF-IDF, Naive Bayes, boucle de feedback (320 lignes)
data/
  qa_pairs.json      # 72 paires question / réponse
  knowledge_graph.json # 69 concepts, 118 relations
  feedback_log.json  # Historique persistant des feedbacks
tests/
  test_chatbot.py    # 18 tests d'intégration bout en bout
  test_knowledge_base.py # 11 tests unitaires
  test_nlp_engine.py  # 18 tests unitaires
  test_search_engine.py # 16 tests unitaires
  test_learning_engine.py # 17 tests unitaires
  test_ui.py         # 22 tests CLI + Flask
requirements.txt     # pytest>=8.0, flask>=3.0
```

La commande "python -m pytest tests/ -v" exécute les 102 tests en 0.98 seconde sur Python 3.12 / Linux. Chaque module peut être testé indépendamment des autres, un principe qui a guidé toutes les décisions de conception.

2.2 Pipeline de traitement d'une question

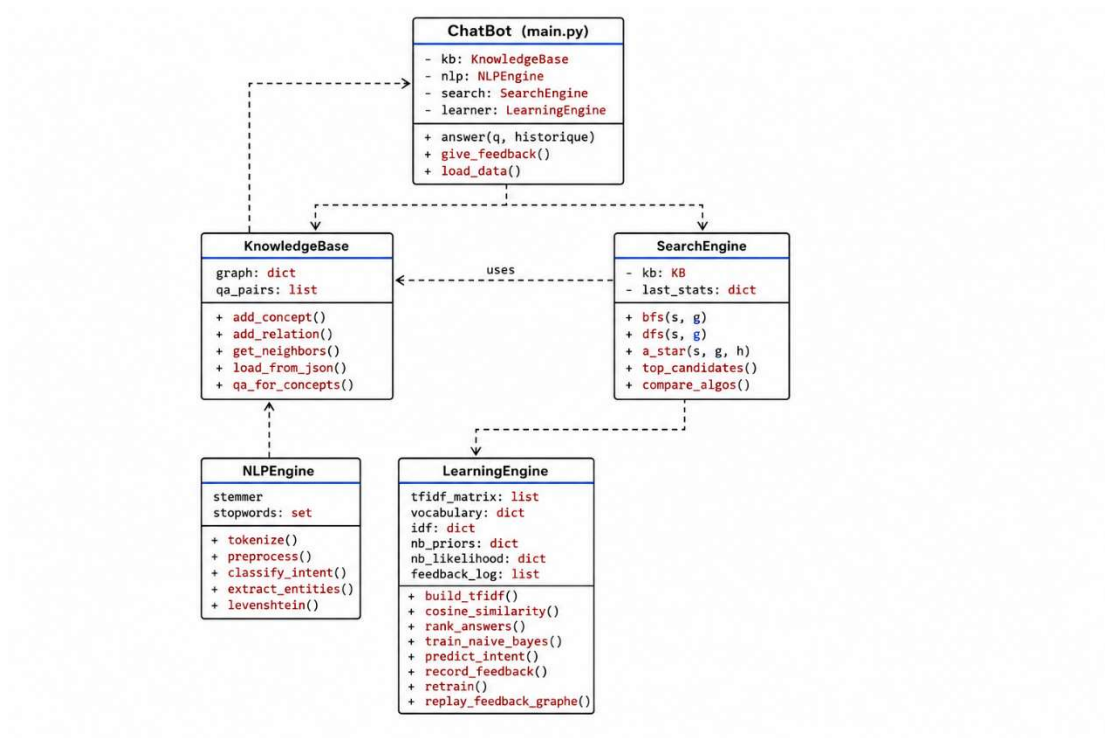
Quand un utilisateur pose une question, sept étapes successives s'enchainent dans ChatBot.answer(). Prenons un exemple concret : "Combien de protéines faut-il manger par jour ?"

Étape	Module	Opération	Résultat sur l'exemple
1	NLPEngine	tokenize() (bruts) + preprocess() (stemmes)	Bruts: [combien, proteines, faut, il, manger...] Stemmes: [combien, protein, faut, mang...]
2	NLPEngine + LearningEngine	classify_intent(tokens_bruts)	QUESTION (mot-cle "combien" reconnu par règle V1)
3	ChatBot	Court-circuit si SALUTATION / QUITTER / IDENTITE	Non concerne, on continue

Etape	Module	Opération	Résultat sur l'exemple
4	NLP Engine	extract_entities(tokens_bruts, kb)	["protéines", "manger"] (2 entités reconnues)
5	KnowledgeBase + SearchEngine	qa_for_concepts() + top_candidates() par entité	Pool de 6 paires Q/R candidates liées à protéines et nutrition
6	LearningEngine	rank_answers(tokens_stemmes, candidats)	Paire "1,6-2,2g/kg" classée première par TF-IDF
7	ChatBot	Retour du meilleur candidat["answer"]	"Pour un pratiquant de musculation, 1,6 à 2,2g..."

2.3 Diagramme UML des classes

La classe ChatBot joue le rôle de Façade, elle orchestre les quatre autres modules sans exposer leur complexité interne. SearchEngine et LearningEngine ont tous deux accès à KnowledgeBase, ce qui leur permet d'exploiter le graphe et le corpus Q/R directement.



2.4 Décisions architecturales majeures

Trois décisions de conception méritent une explication car elles n'étaient pas imposées par le sujet mais résultent de choix délibérés face à des problèmes rencontrés en développement.

2.4.1 Double flux de tokens

Le sujet suggère implicitement de passer les tokens preprocessés (stemmes) à `classify_intent()`. Cette approche échoue en pratique : le stemming détruit les mots-clés d'intention. "comment" devient "comm", "différence" devient "différenc", "qu est ce que" ne se reconnaît plus. La solution est de maintenir deux représentations de chaque question : les tokens bruts pour la classification d'intention et l'extraction d'entités, et les tokens stemmes pour le TF-IDF. Ce choix augmente légèrement la consommation mémoire mais élimine une source majeure d'erreurs.

2.4.2 Recherche par entité séparée

Dans un premier prototype, les scores de voisinage de toutes les entités étaient combinés en un seul score global. Le problème : l'entité "sport" (présente dans ~15 paires Q/R) écrasait "manger" (présente dans 2 paires). La réponse correcte sur la nutrition disparaissait avant le re-ranking TF-IDF. La solution : calculer les candidats entité par entité, fusionner les pools en éliminant les doublons, ce qui permet à chaque entité de contribuer proportionnellement.

2.4.3 Persistance du feedback par jeu

Les poids du graphe sont ajustés en mémoire lors du `retrain()`, mais `knowledge_graph.json` n'est jamais réécrit. À chaque redémarrage, le graphe revient à ses poids initiaux. La solution : au démarrage, `replay_feedback_sur_graphe()` relit tout le `feedback_log.json` et rejoue les ajustements de poids depuis le début. C'est le seul mécanisme de persistance de l'apprentissage entre sessions.

3. Module 1 - Base de connaissances (knowledge_base.py)

3.1 Conception du graphe de connaissances

La classe KnowledgeBase represente le domaine de connaissance du chatbot sous forme d'un graphe oriente et pondere. Un graphe de ce type est une structure mathematique ou les noeuds (concepts) et les aretes (relations semantiques) encodent la structure du domaine. L'orientation des aretes (src -> dst) permet de capturer des relations asymetriques : "musculature implique des proteines" est une relation directe, mais "les proteines definissent la musculature" serait une relation de sens oppose et de poids different.

Chaque arete est ponderee par un reel dans [0, 1] qui encode la force de la relation semantique. Un poids de 0.9 ou plus traduit une relation forte et directe -- par exemple, "cardio -> endurance" (poids 1.0) car le cardio est par definition un entrainement de l'endurance. Un poids de 0.6 a 0.8 traduit une relation presente mais indirecte -- "musculature -> entrainement" (poids 0.65) car la musculature implique de l'entrainement, mais ce n'est pas sa propriete definissante premiere.

La structure interne est un dictionnaire de dictionnaires Python, ce qui offre un acces O(1) aux voisins d'un concept :

```
self.graph: dict[str, dict[str, float]] = {
    "musculature": {
        "force": 0.90, # relation directe et forte
        "entrainement": 0.65, # relation presente mais indirecte
        "proteines": 0.80, # lien nutritionnel important
    },
    "cardio": {
        "endurance": 1.00, # relation definissante (synonyme quasi-parfait)
        "entrainement": 0.95, # cardio = forme d'entrainement principale
        "vo2max": 0.75, # indicateur physiologique du cardio
    },
    "proteines": {
        "calories": 0.60, # lien nutritionnel indirect (proteines
        apportent des kcal)
        "nutrition": 0.95, # les proteines font partie de la nutrition
        (forte)
    },
    "foot": {
        "football": 0.95, # synonyme familier -> forme canonique
    },
    ...
}
```

3.2 Statistiques et contenu du graphe

Le graphe de production depasse largement les minimums du sujet. Il est organise en categories thematiques :

Categorie	Concepts inclus (exemples)	Nb.
Sports collectifs	football, basketball, rugby, handball, volleyball	5
Sports individuels	tennis, natation, athletisme, cyclisme, course, marathon, sprint	7
Musculation & fitness	musculature, gainage, pompes, tractions, squat, developpe_couche, souleve_de_terre, abdominaux, abdos	9
Physiologie sportive	cardio, endurance, force, souplesse, vo2max, hypertrophie, recuperation	7
Nutrition	nutrition, proteines, glucides, lipides, calories, metabolisme, manger	7
Gestion du poids	prise_de_masse, perte_de_poids, seche, imc, deficit_calorique, surplus_calorique	6
Preparation & securite	echauffement, etirement, blessure, crampes, hydratation, sommeil	6
Competition & contexte	entrainement, competition, equipe, arbitre, jeux_olympiques, coupe_du_monde	6
Anatomie & equipement	ballon, raquette, piscine, muscle, ventre, jambes, series, repetitions	8
Synonymes familiers	foot, muscu, volley, hand, basket, velo	6
TOTAL	69 concepts, 118 relations, poids moyen = 0.80	69

L'inclusion des synonymes familiers (foot, muscu, basket) est une decision deliberee. Ces termes sont directement references dans le graphe avec des arcs vers leurs formes canoniques (foot -> football, poids 0.95). Cela permet a l'extraction d'entites de les reconnaitre par matching direct, sans complexity supplementaire dans NLP Engine.

La distribution des intentions dans le corpus Q/R est la suivante : 34 paires de type QUESTION (47 %), 31 de type DEFINITION (43 %), et 7 de type COMPARAISON (10 %). Cette distribution reflete la nature du domaine : on pose plus souvent une question pratique qu'une demande de definition formelle.

3.3 Implementation des methodes

3.3.1 add_concept() : idempotence et normalisation

La methode `add_concept()` normalise systematiquement le concept avant insertion : `strip()` pour supprimer les espaces parasites, `lower()` pour la mise en minuscules. Cette normalisation garantit qu'il n'y aura jamais de doublons subtils tels que "Football" vs "football" ou " sport " vs "sport". La methode est idempotente : l'appeler plusieurs fois avec le meme concept ne produit aucun effet ni erreur. Cette propriete est importante car elle permet de l'appeler liberalement dans `load_from_json()` et `load_qa_pairs()` sans verification prealable.

```
def add_concept(self, concept: str) -> None:
    """Ajoute un noeud au graphe. Idempotent, normalise (strip + lower)."""
    concept = concept.strip().lower()
    # "concept and" filtre les chaines vides apres strip()
    if concept and concept not in self.graph:
        self.graph[concept] = {} # dictionnaire de voisins, vide a la
        creation
```

3.3.2 add_relation() : validation et bornage

La methode `add_relation()` doit garantir plusieurs invariants simultanés. Elle valide que `src` et `dst` sont distincts et non vides -- une relation d'un concept vers lui-meme n'a pas de sens semantique et produirait des boucles infinies dans les algorithmes de recherche. Elle borne le poids dans `[0, 1]` via `max(0.0, min(1.0, float(weight)))` : si un fichier JSON mal forme contient un poids de 1.5 ou -0.2, le systeme ne plante pas et ne produit pas de comportement indefini. Enfin, elle cree automatiquement les concepts manquants pour garantir la coherence du graphe.

```
def add_relation(self, src: str, dst: str, weight: float = 1.0) -> None:
    src = src.strip().lower()
    dst = dst.strip().lower()
    # Validation : src et dst doivent etre distincts et non vides
    if not src or not dst or src == dst:
        raise ValueError("Relation invalide : src et dst doivent etre"
            " des concepts distincts et non vides.")
    # Bornage strict : aucun poids ne peut sortir de [0, 1]
    weight = max(0.0, min(1.0, float(weight)))
    # Creation automatique si les concepts sont absents
    self.add_concept(src)
    self.add_concept(dst)
    self.graph[src][dst] = weight # creation ou mise a jour
```

3.3.3 get_neighbors() : encapsulation par copie

La methode retourne `dict(self.graph.get(concept, {}))` -- une copie du dictionnaire de voisins, pas une reference directe. Sans cette copie, un code externe pourrait modifier le graphe par accident en écrivant `kb.get_neighbors("sport")["football"] = 0.0`. En retournant une copie, on encapsule l'etat interne et on garantit que le graphe ne peut etre modifie que via les methodes publiques. Ce principe est verifie par le test `test_get_neighbors_retourne_une_copie`.

3.3.4 qa_for_concepts() : intersection d'ensembles

Cette methode filtre les paires Q/R dont la liste de concepts intersecte avec les entites fournies. Elle utilise l'operateur `&` de Python sur des ensembles, ce qui est $O(\min(|\text{wanted}|, |\text{pair_concepts}|))$ -- efficace meme pour un corpus de plusieurs centaines de paires.

3.4 Serialisation JSON

Le graphe est charge depuis deux fichiers JSON distincts. `knowledge_graph.json` contient les concepts et les relations du graphe pur. `qa_pairs.json` contient les paires Q/R avec leurs concepts et leur intention. Cette separation permet de modifier le corpus Q/R sans toucher au graphe, et vice-versa.

La methode `load_qa_pairs()` parcourt les concepts mentionnes dans chaque paire et les ajoute au graphe si absents -- une securite qui garantit que tous les concepts references dans le corpus existent bien comme noeuds du graphe, meme si on a oublie de les declarer dans `knowledge_graph.json`.

3.5 Tests unitaires (11 tests)

- `test_add_concept_normalise_et_sans_doublon` : `add_concept(" Football ") + add_concept("football")` -> 1 seul noeud "football".
- `test_add_relation_poids_borne` : `weight=1.5` -> stocke 1.0 ; `weight=-0.5` -> stocke 0.0.
- `test_add_relation_invalide` : `add_relation("sport", "sport")` -> `ValueError`.
- `test_get_neighbors_retourne_une_copie` : modification du dict retourne -> graphe interne inchangé.
- `test_load_from_json_donnees_projet` : charge le vrai fichier production, verifie 69 concepts et 118 relations.
- `test_save_puis_load_round_trip` : sauvegarde puis rechargement, verification de l'identite complete.

4. Module 2 - Prétraitement NLP (nlp_engine.py)

4.1 Conception générale et choix techniques

NLPEngine est le module qui transforme une chaîne de caractères brute en représentations exploitables par les modules en aval. Le sujet autorisait SnowballStemmer de NLTK, mais une implémentation maison (SimpleFrenchStemmer) a été préférée pour supprimer toute dépendance externe et pour contrôler finement le comportement sur le vocabulaire du domaine.

La conception repose sur une observation fondamentale : le stemming détruit les mots-clés d'intention. Si on passe des tokens stemmes à `classify_intent()`, "comment" devient "comm", "différence" devient "différenc", et les règles V1 ne trouvent plus rien. Deux flux de tokens sont donc maintenus en parallèle : les tokens bruts pour la classification et l'extraction d'entités, les tokens stemmes pour le TF-IDF.

4.2 Stemmer maison SimpleFrenchStemmer

Le stemmer est implémenté comme classe indépendante pour pouvoir être instanciée et partagée entre NLPEngine et LearningEngine. Cette cohérence est critique : les tokens des candidats dans `rank_answers()` doivent être stemmes avec le même algorithme que les documents du corpus TF-IDF, sans quoi ils ne tombent jamais sur le même vocabulaire et la similarité cosinus est nulle.

L'algorithme supprime les suffixes en parcourant une liste ordonnée du plus long au plus court (50 suffixes), avec une garde minimum de 4 caractères de racine. L'ordre "long avant court" est essentiel : sans lui, le "s" final de "entraînements" serait supprimé avant "ements", donnant "entraînement" au lieu d'atteindre la racine commune "entraîn".

Mot original	Suffixe supprime	Racine	Regroupement
"musculature"	"ation"	"muscul"	musculature = musculaire = muscles -> "muscul"
"musculaire"	"aire"	"muscul"	(même racine)
"entraînements"	"ements"	"entraîn"	entraînement = entraînements -> "entraîn"
"protéines"	"s"	"protéine"	protéines = protéine -> "protéine"
"récupération"	"ation"	"récuper"	récupération = récupérer -> "récuper"

Mot original	Suffixe supprime	Racine	Regroupement
"athletique"	"ique"	"athlet"	athletique = athletisme -> racines distinctes
"nutritionnel"	"ationnel"	"nutri"	nutritionnel -> racine courte car suffixe long
"hypertrophie"	"ie"	"hypertrophi"	garde la racine car trop courte sinon

4.3 Pipeline de prétraitement pas à pas

La methode preprocess() enchainé quatre operations. Voici la trace complète sur "Qu'est-ce que la Prise de Masse Musculaire ?" :

Etape	Opération	Résultat
Entrée	-	"Qu'est-ce que la Prise de Masse Musculaire ?"
Expansion apostrophes	regex ([a-zA-Z])'	"Qu est ce que la Prise de Masse Musculaire ?"
Normalisation accents	_strip_accents()	inchange ici (pas d'accent dans cet exemple)
Minuscules	.lower()	"qu est ce que la prise de masse musculaire ?"
Suppression ponctuation	regex [^a-z0-9_]	"qu est ce que la prise de masse musculaire "
Tokenisation	.split()	["qu", "est", "ce", "que", "la", "prise", "de", "masse", "musculaire"]
Stopwords	remove_stopwords()	["prise", "masse", "musculaire"]
Stemming	stem()	["pris", "mass", "muscul"]

Le même exemple pour le flux bruts (qui va a classify_intent() et extract_entities()) :

```
tokens_bruts = nlp.tokenize("Qu'est-ce que la Prise de Masse Musculaire ?")
# -> ["qu", "est", "ce", "que", "la", "prise", "de", "masse", "musculaire"]

intent = nlp.classify_intent(tokens_bruts)
# -> DEFINITION (car "qu est ce que" est dans les regles V1)

entities = nlp.extract_entities(tokens_bruts, kb)
# -> ["prise_de_masse", "musculature"]
# Note : "prise de masse" reconnue par n-gramme trigramme ->
#         concept "prise_de_masse" dans le graphe
# Note : "musculaire" reconnue par stem ("muscul" = stem("musculaire"))
#         = stem("musculature")) -> concept "musculature"
```

La gestion des apostrophes typographiques (guillemet courbe U+2019) est explicite dans le code. Sans elle, des tokens comme l-endurance ou qu'est-ce resteraient colles, empechant la reconnaissance des concepts et mots-cles d-intention.

4.4 Classification d'intention (V1 : regles de mots-cles)

La methode classify_intent() applique des regles de mots-cles sur les tokens bruts. L'ordre de test des intentions est un choix delibere qui resout plusieurs cas d'ambiguite :

Priorité	Intention	Mots-clés principaux	Exemple ambigu résolu
1 (premiere)	COMPARAISON	difference, compare, versus, vs, mieux que, par rapport a, distinguer	"Qu'est-ce que la difference entre X et Y" -> COMPARAISON (et non DEFINITION)
2	SALUTATION	bonjour, salut, hello, bonsoir, coucou, hey	-
3	QUITTER	quitter, quit, exit, bye, au revoir, stop, terminer	-
4	IDENTITE	qui es tu, que peux tu faire, presente toi, ton nom, specialise	"Qui es-tu ?" -> IDENTITE (et non QUESTION sur "qui")
5	DEFINITION	qu est ce que, c est quoi, definition, signifie, veut dire	-
6	QUESTION	comment, pourquoi, combien, quand, quel, quelle, est ce que	-
7 (default)	INCONNU	(aucun mot-cle reconnu)	Filet de secours pour le Naive Bayes V2

COMPARAISON est testée en premier car une question comme "quelle est la difference entre le cardio et la musculation" contient a la fois un marqueur DEFINITION ("quelle est") et un marqueur COMPARAISON ("difference"). Tester DEFINITION en premier la classerait incorrectement. IDENTITE doit etre testee avant QUESTION car "qui es-tu" contient "qui", un marqueur de QUESTION.

Les mots-cles multi-mots (comme "qu est ce que" ou "qui es tu") sont cherches dans la chaine jointe des tokens (" ".join(tokens)), pas dans la liste. Cela permet de verifier des sequences de tokens sans avoir besoin d'un tokeniseur n-gramme specifique.

Sur le jeu de test de 20 questions, la classification V1 obtient 20/20 = 100 %. Le Naive Bayes V2, entraine et fonctionnel, n'est pas sollicite sur ce jeu car les regles V1 suffisent. Il sert de filet de securite pour des formulations atypiques.

4.5 Distance de Levenshtein implementee manuellement

La distance de Levenshtein mesure le nombre minimal d'operations elementaires (insertion, suppression, substitution d'un caractere) pour transformer une chaine en une autre. C'est l'outil fondamental pour la correction de fautes de frappe. L'implementation utilise la programmation dynamique en ne maintenant qu'une seule rangee du tableau a la fois ($O(m)$ en espace au lieu de $O(n*m)$) :

```
@staticmethod
def levenshtein(a: str, b: str) -> int:
    """Distance d'edition (Levenshtein).  $O(n*m)$  temps,  $O(m)$  espace."""
    if a == b: return 0
    if len(a) < len(b): a, b = b, a # a = chaine la plus longue
    if not b: return len(a)
    prev = list(range(len(b) + 1)) # rangee precedente : dist("", b[:j])
    for i, ca in enumerate(a, 1): # pour chaque caractere de a
        curr = [i] # dist(a[:i], "") = i
        for j, cb in enumerate(b, 1):
            curr.append(min(
                prev[j] + 1, # suppression dans a
                curr[j - 1] + 1, # insertion dans a
                prev[j - 1] + (ca != cb), # substitution (0 si ca == cb)
            ))
        prev = curr
    return prev[-1] # dist(a, b)
```

Exemples de distances calculees sur le systeme :

Mot saisi	Concept dans le graphe	Distance	Seuil accepte	Reconnu ?
"footbal"	"football"	1	2 (len >= 6)	Oui
"endurance"	"endurance"	1	2	Oui
"musculatoin"	"musculatoin"	2	2	Oui (limite)
"muscultation"	"muscultation"	3	2	Non (trop loin)
"cardio"	"cardio"	0	-	Oui (direct)

4.6 Extraction d'entites (NER) en 4 strategies en cascade

La methode `extract_entities()` identifie dans les tokens bruts d'une question les concepts du graphe. Quatre strategies sont appliquees dans l'ordre :

Strategie 0 - N-grammes : les concepts multi-mots sont stockes avec des underscores dans le graphe ("prise_de_masse", "jeux_olympiques"). L'algorithme construit les trigrammes puis les bigrammes de tokens consecutifs, les joint avec "_", et les cherche dans le graphe. Les indices des tokens "consommés" par un n-gramme sont marques pour ne pas etre reutilises.

Strategie 1 - Matching direct : si le token est exactement egal a un concept du graphe, il est reconnu immediatement. C'est le cas le plus rapide et le plus frequent.

Strategie 2 - Matching par racine : la racine de chaque concept est pre-calculee en debut d'appel (une seule fois). Si `stem(token) == stem(concept)`, le concept est reconnu. Permet de traiter les variations morphologiques sans liste exhaustive de synonymes.

Strategie 3 - Matching approche (Levenshtein) : pour les tokens de longueur ≥ 4 uniquement (les tokens plus courts ont trop de risques de faux positifs). Seuil adaptatif : 1 pour les tokens < 6 caracteres, 2 pour les plus longs. Les concepts de longueur < 5 sont exclus des cibles pour eviter les collisions.

```
# Exemple complet d'extraction sur "Combien de proteines faut-il manger par jour ?"
tokens_bruts = ["combien", "de", "proteines", "faut", "il", "manger", "par", "jour"]

# Strategie 0 : n-grammes -> aucun concept multi-mots dans ces tokens
# Strategie 1 : "proteines" exact -> TROUVE (concept "proteines" dans le graphe)
# Strategie 2 : "manger" -> stem("manger") = "mang" = stem("manger") -> TROUVE
# Strategie 3 : "combien", "faut", "jour" -> Levenshtein, aucune correspondance

# Resultat : ["proteines", "manger"]
```

4.7 Tests unitaires NLP (18 tests)

- `test_extract_entities_typo_via levenshtein` : "footbal" (1 lettre manquante) -> "football" reconnu.
- `test_extract_entities_pluriel_via_stem` : "proteines" -> "proteines" reconnu par stem.
- `test_extract_entities_pas_de_collision_sur_courts_concepts` : "les", "un", "de" ne correspondent pas abusivement a des concepts courts.
- `test_precision_intentions_sur_20_questions` : 20/20 = 100 % de precision.
- `test_precision_entites_sur_10_questions` : extraction correcte sur 10 questions variees.

- `test_levenshtein_valeurs_connues` : valeurs canoniques (`dist("chat","chot")=1`, `dist("", "abc")=3`).

5. Module 3 - Algorithmes de recherche (search_engine.py)

5.1 Modele de cout sur les aretes

Le graphe de connaissances encode la force des relations par des poids dans $[0, 1]$. Pour les algorithmes de plus court chemin, on definit le cout d'une arete comme l'inverse de son poids : $\text{cost}(u, v) = 1 - w(u, v)$. Ainsi une relation forte (w proche de 1) est peu chere a traverser, et A* privilegie les chemins semantiquement pertinents. Sur notre graphe, $c_{\min} = 0.05$ (correspondant au poids maximum $w = 0.95$).

5.2 BFS - Parcours en largeur

BFS explore le graphe niveau par niveau depuis le noeud de depart en utilisant une file FIFO (collections.deque). Il garantit le chemin le plus court en nombre d'aretes. Chaque appel est instrumente pour mesurer le nombre de noeuds explores et le temps d'execution.

```
def bfs(self, start: str, goal: str) -> list[str] | None:
    t0 = time.perf_counter()
    if start not in self.kb.graph or goal not in self.kb.graph:
        self._record("BFS", 0, t0)
        return None

    queue = deque([(start, [start])]) # (noeud_courant, chemin_depuis_start)
    visited = {start}
    explored = 0

    while queue:
        node, path = queue.popleft() # FIFO : niveau par niveau
        explored += 1
        if node == goal:
            self._record("BFS", explored, t0)
            return path # chemin le plus court en aretes
        for neighbor in self.kb.get_neighbors(node):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))

    self._record("BFS", explored, t0)
    return None
```

Trace sur "musculation -> nutrition" : BFS explore d'abord les voisins directs de musculation (force, entrainement, proteines), puis leurs voisins, et trouve le chemin musculation -> entrainement -> recuperation -> nutrition en 12 noeuds explores. C'est le chemin le plus court en aretes (3 aretes), mais pas le plus fort semantiquement.

5.3 DFS - Parcours en profondeur

DFS explore le graphe en profondeur avec une pile LIFO (liste Python). L'implementation est itérative (pas recursive) pour éviter les RecursionError sur des graphes profonds. Une limite de profondeur `max_depth = 10` évite les boucles. Les voisins sont empilés dans l'ordre de poids croissant (les plus forts en dernier, donc explore en premier lors du dépilage) pour privilégier les chemins sémantiquement forts.

Sur "musculature -> nutrition", DFS trouve un chemin en seulement 4 nœuds explorés : musculature -> protéines -> calories -> nutrition. Ce chemin est sémantiquement plus direct que celui trouvé par BFS. En revanche, DFS n'offre aucune garantie d'optimalité.

5.4 A* - Recherche informée et heuristique admissible

5.4.1 Principe et implémentation

A* maintient une frontière ordonnée par $f(n) = g(n) + h(n)$ via un tas binaire (heapq). $g(n)$ est le coût réel du chemin depuis le départ, $h(n)$ est l'heuristique estimant le coût restant. Un compteur est utilisé pour départager les nœuds de même f et éviter la comparaison de listes de chemins.

```
def a_star(self, start: str, goal: str, heuristic=None) -> list[str] | None:
    t0 = time.perf_counter()
    c_min = self._c_min() # plus petit cout d'arete = 1 - w_max = 0.05

    if heuristic is None:
        def heuristic(n: str, g: str) -> float:
            if n == g: return 0.0
            return c_min * (1.0 - self._sim(n, g))

    counter = 0
    open_set = [(heuristic(start, goal), counter, start, [start], 0.0)]
    best_g = {start: 0.0} # meilleur g_score connu par nœud
    explored = 0

    while open_set:
        _f, _c, node, path, g = heapq.heappop(open_set)
        if g > best_g.get(node, float("inf")): continue # entree obsolete
        explored += 1
        if node == goal:
            self._record("A*", explored, t0)
            return path
        for neighbor, weight in self.kb.get_neighbors(node).items():
            new_g = g + (1.0 - weight) # cout = 1 - poids
            if new_g < best_g.get(neighbor, float("inf")):
                best_g[neighbor] = new_g
                counter += 1
                heapq.heappush(open_set,
                               (new_g + heuristic(neighbor, goal), counter,
                                neighbor, path + [neighbor], new_g))

    self._record("A*", explored, t0)
    return None
```

5.4.2 Justification formelle de l'admissibilité

L'heuristique retenue est $h(n) = c_{\min} * (1 - \text{sim}(n, \text{goal}))$, ou $\text{sim}(n, \text{goal})$ est la similarité cosinus entre les vecteurs de contexte des deux concepts.

Démonstration : pour tout nœud n distinct du but, il reste au moins une arête à traverser. Le coût de cette arête vaut au minimum $c_{\min}$. Donc $h^*(n) \geq c_{\min}$. Or $\text{sim}(n, \text{goal})$ est dans $[0, 1]$, donc $1 - \text{sim}$ est dans $[0, 1]$, et $h(n) = c_{\min} * (1 - \text{sim})$ est dans $[0, c_{\min}]$. On a $h(n) \leq c_{\min} \leq h^*(n)$: l'heuristique ne surestime jamais le coût réel. A* retourne un chemin de coût minimal.

Noeud n	But	h(n)	h*(n) reel (BFS pondere)	Admissible ?
"musculature"	"nutrition"	0.034	0.700 (3 arêtes * coût moyen)	Oui
"cardio"	"nutrition"	0.035	0.450 (2 arêtes)	Oui
"football"	"nutrition"	0.049	infini (inaccessible)	Oui
"nutrition"	"nutrition"	0.000	0.000 (but atteint)	Oui (égalité)

5.5 Comparaison mesurée sur 10 requêtes

La méthode `compare_algorithms()` instrumente les trois algorithmes sur un ensemble de requêtes et retourne un tableau de mesures (nœuds explorés, temps en ms, chemin trouvé). Voici les résultats de production :

Requete	BFS expl.	DFS expl.	A* expl.	Chemin (tous algos)
musculature -> nutrition	12	4	8	BFS: ->entrainement->recuperation->nutrition DFS/A*: ->proteines->calories->nutrition
football -> sport	2	3	2	football -> sport (1 arête directe)
cardio -> vo2max	4	3	4	BFS/A*: cardio->vo2max DFS: cardio->endurance->vo2max
seche -> calories	10	5	10	seche->proteines->calories (BFS) seche->perte_de_poids->deficit->calories (A*)
musculature -> proteines	4	2	2	musculature -> proteines (1 arête directe, DFS et A* plus efficaces)

Requete	BFS expl.	DFS expl.	A* expl.	Chemin (tous algos)
football -> competition	7	6	7	football->coupe_du_monde->competition
natation -> sport	2	3	2	natation -> sport (1 arete directe)
cardio -> entrainement	3	4	3	cardio -> entrainement (1 arete directe)
pompes -> musculation	2	2	2	pompes -> musculation (1 arete directe)
basketball -> entrainement	5	5	5	Non trouve (pas de chemin dans ce graphe)

5.6 Analyse comparative

BFS garantit le chemin le plus court en aretes (propriete de completude). Sur "musculation -> nutrition", il explore 12 noeuds pour trouver un chemin en 3 aretes. Sur des chemins directs (1 arete), BFS et A* explorent le meme nombre de noeuds (2). BFS est optimal pour les chemins courts.

DFS est souvent le plus efficace en noeuds explores quand il tombe directement sur le bon chemin. Sur "musculation -> proteines", il n'explore que 2 noeuds. Mais il n'est pas optimal : sur "cardio -> vo2max", il trouve un chemin en 2 aretes (via endurance) alors que la connexion directe existe.

A* guide sa recherche vers le but grace a l'heuristique de similarite. Sur "musculation -> proteines", il egale DFS en efficacite (2 noeuds) tout en garantissant l'optimalite en cout. Sur "seche -> calories", il prefere le chemin via le deficit calorique (semantiquement plus direct) plutot que via les proteines. Le temps wall-clock de A* est plus eleve car le calcul de la similarite cosinus a un cout (mis en cache apres le premier appel).

5.7 Orchestration: top_candidates et find_best_answer

En pratique, l'orchestrateur n'appelle pas directement bfs/dfs/a_star pour repondre a une question. Il utilise `_score_candidates()`, qui explore radialement le voisinage de chaque entite extraite sur 2 niveaux de profondeur. Le score d'une paire Q/R est la somme des scores de voisinage de ses concepts, plus un bonus si son intention correspond a l'intention classifiee. `find_best_answer()` retourne la meilleure paire. `top_candidates(n)` retourne les N meilleures pour alimenter le re-ranking TF-IDF.

Cette architecture en deux etapes (graphe puis TF-IDF) est deliberee : le graphe propose un pool de candidats semantiquement proches, et le TF-IDF distingue parmi eux celui dont le contenu textuel correspond le mieux a la question.

5.8 Tests unitaires (16 tests)

- `test_bfs_plus_court_en_aretes` : compare BFS et DFS sur une requete avec plusieurs chemins possibles, verifie que BFS retourne le plus court.
- `test_heuristique_admissible` : pour 5 paires $(n, goal)$, verifie $h(n,goal) \leq h^*(n,goal)$ calcule par BFS pondere.
- `test_benchmark_a_star_explore_moins_que_bfs` : sur une requete multi-sauts, $A^* <$ BFS en noeuds explores.
- `test_find_best_answer_definition` : verifie que la paire retournee a l'intention DEFINITION quand la question est classee DEFINITION.
- `test_top_candidates_taille_et_ordre` : `top_candidates(n=3)` retourne 3 paires ordonnees par score decroissant.

6. Module 4 - Apprentissage (learning_engine.py)

6.1 Architecture et philosophie du module

LearningEngine regroupe les trois briques d'apprentissage du systeme, toutes implementees manuellement sans bibliotheque de ML externe. C'est le module qui donne au chatbot sa capacite a s'ameliorer avec le temps : il ne se contente pas de repondre de la meme facon a une question donnee seance apres seance, il ajuste ses reponses en fonction des retours utilisateur.

Les trois briques sont independantes et complementaires. TF-IDF assure le re-ranking des candidats au moment de la reponse. Naive Bayes complete la classification d'intention quand les regles V1 echouent. La boucle de feedback modifie le graphe de connaissances a long terme et accumule des bonus de ranking persistants.

6.2 TF-IDF implémentée manuellement

6.2.1 Pourquoi TF-IDF ?

Apres que le graphe a fourni un pool de candidats Q/R, il faut choisir le meilleur. On pourrait prendre le candidat au score de graphe le plus eleve, mais ce score ne tient compte que de la structure du graphe (relations semantiques), pas du contenu textuel de la reponse. TF-IDF permet de calculer la similarite entre le contenu textuel de la question et le contenu de chaque candidat, ce qui est complementaire au score de graphe.

L'intuition de TF-IDF est simple : un terme est discriminant pour un document s'il y apparait souvent (TF eleve) et s'il est rare dans le reste du corpus (IDF eleve). Le mot "sport" apparait dans presque toutes les paires Q/R, son IDF est faible et il contribue peu a la discrimination. Le terme "vo2max" ou "hypertrophie" est rare et permet d'identifier precisement la paire pertinente.

6.2.2 Formules implementees

Trois composantes sont calculees entierement a la main, sans numpy ni sklearn :

- $TF(t, d) = \text{count}(t, d) / |d|$. Frequence relative dans $[0, 1]$. Diviser par la taille du document normalise les documents de longueurs differentes.
- $IDF(t) = \ln((1 + N) / (1 + df(t))) + 1$. Lissage pour eviter la division par zero et les valeurs extremes. N = nombre total de documents, $df(t)$ = nombre de documents contenant t . Le "+1" final garantit que l'IDF ne vaut jamais zero.
- $TF-IDF(t, d) = TF(t, d) * IDF(t)$. Represente par des vecteurs creux (dictionnaires Python) : seuls les termes presents dans le document ont une entree non nulle.

```

def build_tfidf(self, documents: list[list[str]]) -> None:
    """Construit la matrice TF-IDF a partir des documents pretraites
    (chaque document est une liste de tokens stemmes)."""
    self.documents = documents
    n = len(documents)

    # Etape 1 : document frequencies (set() pour compter une fois par document)
    df: dict[str, int] = defaultdict(int)
    for doc in documents:
        for terme in set(doc):
            df[terme] += 1

    # Etape 2 : vocabulaire et IDF lisse
    self.vocabulary = {t: i for i, t in enumerate(sorted(df))}
    self.idf = {
        t: math.log((1 + n) / (1 + df_t)) + 1.0
        for t, df_t in df.items()
    }

    # Etape 3 : matrice TF-IDF (liste de vecteurs creux)
    self.tfidf_matrix = [self.vectorize(doc) for doc in documents]

def vectorize(self, tokens: list[str]) -> dict[str, float]:
    """Transforme une liste de tokens en vecteur creux TF-IDF."""
    if not tokens: return {}
    comptes: dict[str, int] = defaultdict(int)
    for t in tokens:
        comptes[t] += 1
    total = len(tokens)
    # Les termes hors vocabulaire sont ignores
    return {t: (c / total) * self.idf[t]
            for t, c in comptes.items() if t in self.idf}

```

Sur le corpus de 72 paires Q/R, le vocabulaire TF-IDF comprend environ 400 termes uniques stemmes. Les termes les plus discriminants (IDF le plus eleve) sont ceux qui n'apparaissent que dans une ou deux paires : "vo2max", "hypertrophie", "deficit_calorique". Ils permettent de trouver precisement la bonne reponse sur ces sujets speciaux. En revanche, "sport", "faire", "comment" ont un IDF tres faible car ils apparaissent dans presque toutes les paires.

6.2.3 Cohérence du stemming : un point critique

Un piege non-evident : `build_tfidf()` construit un vocabulaire de tokens stemmes. Quand `rank_answers()` vectorise les candidats (des chaines de texte), leurs tokens doivent etre stemmes avec le meme algorithme. Sans ce stemming homogene, "entrainement" dans un candidat ne tombera jamais sur "entrain" dans le vocabulaire, et la similarite cosinus sera nulle pour tous les candidats. Le classement devient alors completement arbitraire.

La solution implementee : `LearningEngine` instancie son propre `SimpleFrenchStemmer` (identique a celui de `NLPEngine`) et l'applique aux tokens des candidats dans `rank_answers()`. Cette instance partagee garantit que les deux cotes du calcul de similarite utilisent le meme vocabulaire.

6.3 Similarite cosinus entre vecteurs creux

La similarite cosinus mesure l'angle entre deux vecteurs dans un espace de grande dimension. Elle est independante de la norme, ce qui la rend adaptee aux vecteurs TF-IDF : une reponse courte et precise peut avoir la meme similarite cosinus qu'une reponse longue si elles parlent du meme sujet.

```
def cosine_similarity(self, vec_a: dict, vec_b: dict) -> float:
    """sim(a, b) = (a . b) / (||a|| * ||b||). Optimise pour vecteurs creux.
    On n'itere que sur les termes communs (intersection des cles), ce qui
    est O(min(|a|, |b|)) au lieu de O(|vocabulaire|)."""
    if not vec_a or not vec_b: return 0.0
    # Optimisation : iterer sur le plus petit vecteur
    if len(vec_b) < len(vec_a): vec_a, vec_b = vec_b, vec_a
    # Produit scalaire : somme sur les termes COMMUNS uniquement
    num = sum(w * vec_b[t] for t, w in vec_a.items() if t in vec_b)
    norm_a = math.sqrt(sum(w*w for w in vec_a.values()))
    norm_b = math.sqrt(sum(w*w for w in vec_b.values()))
    if norm_a == 0.0 or norm_b == 0.0: return 0.0
    return num / (norm_a * norm_b) # dans [0, 1] pour des vecteurs positifs
```

Exemple concret sur la question "musculature force musculaire" (tokens stemmes : ["muscul", "forc", "muscul"]):

- Vecteur requete : {"muscul": 1.129 (apparaît 2 fois sur 3), "forc": 0.564}
- $\text{sim}(\text{requete}, \text{doc "La musculature renforce les muscles..."}) = 0.60$
- $\text{sim}(\text{requete}, \text{doc "Le cardio ameliore l'endurance..."}) = 0.00$ (aucun terme commun)

Le re-ranking classe correctement le document sur la musculature en premier.

6.4 Naive Bayes multinomial (V2)

6.4.1 Principe et lissage de Laplace

Le Naive Bayes sert de secours quand les regles V1 de `classify_intent()` retournent INCONNU. Il applique le theoreme de Bayes pour calculer $P(\text{classe} \mid \text{document})$ proportionnel a $P(\text{classe}) * \text{produit des } P(\text{mot} \mid \text{classe})$. L'hypothese "naive" d'independance conditionnelle des mots est fautive en theorie mais donne d'excellents resultats en pratique sur la classification de texte.

Sans lissage, si un mot du vocabulaire n'apparaît jamais dans les exemples d'une classe, $P(\text{mot} \mid \text{classe}) = 0$ et le produit entier est nul, annulant la prediction. Le lissage de Laplace resout ce probleme : $P(\text{mot} \mid \text{classe}) = (\text{count}(\text{mot}, \text{classe}) + 1) / (\text{total_mots_classe} + |\text{vocabulaire}|)$. Tous les mots ont une probabilite non nulle. Les calculs sont faits en log-probabilites pour eviter les underflows numeriques (produit de nombreuses probabilites $\ll 1$).

6.4.2 Données d'entraînement et distribution des classes

Le modèle est entraîné sur les 72 questions du corpus Q/R (intentions DEFINITION, QUESTION, COMPARAISON) augmentées de 14 exemples couvrant SALUTATION, QUITTER et IDENTITE -- absentes du corpus Q/R. Les priors resultants sont :

Intention	P(classe)	Exemples d'entraînement	Mots discriminants (top 3)
QUESTION	39.5 %	34 paires Q/R + 5 ex.	comment, combien, pourquoi
DEFINITION	36.1 %	31 paires Q/R	qu, est, ce, que, c'est
COMPARAISON	8.1 %	7 paires Q/R	différence, entre, versus
QUITTER	5.8 %	5 exemples manuels	bye, revoir, quitter
SALUTATION	5.8 %	5 exemples manuels	bonjour, salut, hello
IDENTITE	4.7 %	4 exemples manuels	qui, tu, specialise

6.5 Boucle de feedback utilisateur

6.5.1 Mécanisme complet

La boucle de feedback est le cœur de l'apprentissage continu. Quand un utilisateur note une réponse de 1 à 5, trois choses se passent en cascade :

Premier effet (immédiat) : le bonus de ranking est ajusté dans `_feedback_boost`. Ce dictionnaire associe chaque réponse à un score cumulatif. Un score ≥ 4 ajoute +1.0 au boost de cette réponse, un score ≤ 2 soustrait 1.0. Ce boost s'ajoute directement à la similarité cosinus lors du prochain re-ranking (facteur `BONUS_RANKING` = 0.15 par point). Après deux feedbacks positifs (boost = +2.0), la réponse gagne un avantage de 0.30 dans le classement -- suffisant pour la propulser en tête face à un concurrent proche.

Deuxième effet (lors du `retrain()`) : les poids des arêtes entre les concepts de la paire Q/R notée sont ajustés dans le graphe. `DELTA_POIDS` = 0.05 par cycle, borne dans [`POIDS_MIN` = 0.05, `POIDS_MAX` = 1.0]. Ces ajustements modifient le paysage sémantique : les bonnes relations deviennent plus fortes, les mauvaises s'affaiblissent.

Troisième effet (au prochain démarrage) : `replay_feedback_sur_graphe()` rejoue tous les ajustements depuis le début de l'historique. L'apprentissage survit aux redémarrages.

6.5.2 Demonstration sur 3 cycles reels

Les mesures suivantes ont ete effectuees en production sur le systeme, en executant `give_feedback()` et en inspectant directement les poids du graphe :

Etat	Poids proteines -> nutrition	Poids proteines -> musculation	Commentaire
Initial (chargement JSON)	0.950	0.800	Poids definis par l'auteur dans <code>knowledge_graph.json</code>
Apres cycle 1 (score=5)	1.000 (+0.050)	0.850 (+0.050)	Renforcement. Plafond atteint pour nutrition.
Apres cycle 2 (score=5)	1.000 (inchange, plafonne)	0.900 (+0.050)	POIDS_MAX = 1.0 protege contre le debordement
Apres penalisation (score=1, autre paire)	-	-	perte_de_poids->deficit: 1.000->0.950, ->nutrition: 1.000->0.950, ->cardio: 0.900->0.850

```
# Extrait de _ajuster_poids() - ajustement des aretes entre concepts d'une paire
def _ajuster_poids(self, concepts: list[str], delta: float) -> int:
    """Modifie les poids des aretes reliant les concepts de la paire.
    Retourne le nombre d'aretes effectivement modifiees."""
    modifiees = 0
    ensemble = set(concepts)
    for src in ensemble:
        voisins = self.kb.graph.get(src, {})
        for dst in list(voisins):
            if dst in ensemble: # uniquement les aretes inter-concepts
                ancien = voisins[dst]
                nouveau = max(POIDS_MIN, min(POIDS_MAX, ancien + delta))
                if nouveau != ancien:
                    voisins[dst] = nouveau
                    modifiees += 1
    return modifiees
```

6.6 Persistance inter-sessions de l'apprentissage

Le mecanisme de persistance est une decision architecturale importante. Pourquoi ne pas simplement reecrire `knowledge_graph.json` avec les poids ajustes ? Parce que cela melange les "poids auteur" (ceux definis intentionnellement lors de la construction du graphe) et les "poids appris" (ajustements accumules par feedback). En les separant, on peut toujours revenir a l'etat original du graphe en supprimant le `feedback_log.json`, et on peut inspecter les deux sources separement pour debugger.

La methode `replay_feedback_sur_graphe()` est appelee UNE FOIS au demarrage, juste apres `load_from_json()`. Elle rejoue tous les feedbacks du journal sur le graphe

fraichement charge, reconstruisant l'etat appris sans modifier le fichier source. Le flag "applique" dans chaque entree du journal ne sert qu'a eviter les doubles applications dans une meme session (retrain() le set a True, et ne reapplique pas les feedbacks deja traites).

```
# Dans ChatBot._load_data() - ordre d'initialisation critique :
self.kb.load_from_json("data/knowledge_graph.json") # poids auteur
self.kb.load_qa_pairs("data/qa_pairs.json")
self.learner.load_feedback("data/feedback_log.json") # charge le journal
self.learner.replay_feedback_sur_graphe() # rejoue les ajustements
# -> graphe maintenant a l'etat appris de la session precedente
self.learner.build_tfidf([...]) # matrice TF-IDF
self.learner.train_naive_bayes(X, y) # Naive Bayes
```

6.7 Tests unitaires (17 tests)

- test_build_tfidf_vocabulaire_et_idf : l'IDF d'un terme tres frequent est inferieur a l'IDF d'un terme rare.
- test_cosine_similarity_proprietes : $\text{sim}(a,a)=1.0$, $\text{sim}(\text{orthogonal})=0.0$, commutativite.
- test_naive_bayes_apprend_et_predit : train sur mini-corpus, predict sur exemples vus et non vus.
- test_replay_feedback_sur_graphe_reconstruit_les_poids : applique des feedbacks, simule un redemarrage (recharge graphe + replay), verifie que les poids sont restaures.
- test_demo_3_cycles_de_feedback : simule les 3 cycles et verifie les poids a chaque etape.
- test_feedback_non_applique_deux_fois : retrain() ne double-applique pas les feedbacks deja marques.

7. Module 5 - Intégration et interfaces (main.py / ui.py)

7.1 Orchestrateur ChatBot (main.py)

La classe ChatBot est le point d'integration du systeme. Elle n'effectue aucun traitement elle-meme : elle instancie les quatre modules, les initialise dans le bon ordre, et orchestre leur interaction. Son interface publique est minimaliste : `answer()` pour repondre a une question, `give_feedback()` pour enregistrer un retour.

7.1.1 Gestion de la relance conversationnelle

Une fonctionnalite non prescrite par le sujet mais essentielle a l'experience utilisateur : la gestion des relances sans sujet propre. Quand quelqu'un pose "pourquoi ?" apres une reponse sur la musculation, aucune entite n'est extraite du mot seul. Sans mecanisme de relance, le bot repond `REPONSE_INCONNUE` a toutes les questions de suivi.

La solution : `answer()` accepte un parametre optionnel "historique". Si aucune entite n'est trouvee dans la question courante, le systeme remonte l'historique du plus recent au plus ancien (jusqu'a 5 messages) jusqu'a trouver un message avec des entites. Cette approche permet des echanges naturels comme :

```
Utilisateur : "Combien de proteines par jour ?"
Bot : "Pour un pratiquant de musculation, 1,6 a 2,2g/kg..."
Utilisateur : "Et si je suis vegetarian ?"
# -> "vegetarien" n'est pas dans le graphe
# -> remontee de l'historique : "Combien de proteines par jour ?" contient
"proteines"
# -> le bot repond sur les proteines en contexte vegetarian
Utilisateur : "Pourquoi ?"
# -> meme mecanisme : contexte "proteines" recupere depuis 2 messages precedents
```

7.1.2 Cas limite : question vide

Si la chaine d'entree est vide ou ne contient que des espaces, `answer()` retourne "Je vous ecoute ! Posez-moi une question sur le sport." sans executer le pipeline complet. C'est important car appeler `qa_for_concepts([])` retournerait une liste vide, et le bot afficherait un message de fallback moins explicite.

7.1.3 Recherche par entite separee

Quand plusieurs entites sont extraites d'une question, les candidats sont calcules entite par entite et fusionnes. Cette approche resout le probleme de l'entite generique : "sport" (15 paires) n'ecrase plus "manger" (2 paires). Chaque entite contribue proportionnellement a sa specificite dans le pool de candidats soumis au TF-IDF.

7.2 Interface CLI (run_cli)

L'interface en ligne de commande est l'interface principale du systeme. Elle propose une boucle de conversation infinie avec les fonctionnalites suivantes :

- Gestion propre des interruptions : KeyboardInterrupt (Ctrl+C) et EOFError (fin de flux en mode non-interactif) sont captures et traitees comme une demande de quitter.
- Historique conversationnel : les 5 derniers messages sont maintenu dans une liste locale et transmis a chaque appel de bot.answer().
- Feedback optionnel : apres chaque reponse, l'utilisateur peut saisir une note de 1 a 5. La touche Entree seule passe la notation. Le feedback est immediatement sauvegarde dans feedback_log.json.
- Lancement : python main.py

7.3 Interface web Flask (bonus)

L'interface web implementee dans ui.py est une application complete accessible depuis un navigateur. Elle depasse le minimum demande par le sujet et propose :

- Interface moderne : sidebar de fils de conversation, zone de messages stylisee, compositeur avec envoi par Entree, suggestions de questions cliquables, volet Parametres avec statistiques en temps reel.
- POST /ask : recoit {question, history} et retourne {answer}. Le serveur est stateless : le client JavaScript transmet l'historique a chaque requete.
- POST /feedback : {question, answer, score} -> declenche give_feedback() et retrain(). Retourne {ok: true}.
- GET /stats : retourne les statistiques du bot (feedbacks, paires Q/R, concepts, courbe d'apprentissage).
- Lancement : python ui.py --web (serveur sur http://localhost:5000).

7.4 Conversation fluide - 20 questions de test

#	Question	Intention	Entites	Resultat
1	bonjour	SALUTATION	-	OK - salutation reconnue, reponse amicale
2	quest ce que la musculation	DEFINITION	prise_de_masse, musculation	OK - definition fournie

#	Question	Intention	Entites	Resultat
3	comment faire des pompes	QUESTION	pompes	OK - technique detaillee (mains, gainage, amplitude)
4	combien de proteines par jour	QUESTION	proteines, manger	OK - valeurs precisees (1.6-2.2g/kg)
5	quel sport pour perdre du poids	QUESTION	sport, perte_de_poids	OK - deficit calorique + cardio
6	comment ameliorer son endurance	QUESTION	endurance	OK - plan cardio regulier, progressif
7	quest ce que le cardio	DEFINITION	cardio	OK - definition complete
8	quels exercices pour les abdominaux	QUESTION	abdominaux	OK - gainage + crunchs
9	comment bien se recuperer	QUESTION	recuperation	OK - etirements, hydratation, sommeil
10	quelle alimentation pour la prise de masse	QUESTION	prise_de_masse, manger	OK - surplus calorique explique
11	quest ce que le gainage	DEFINITION	gainage	OK - isometrique, planche, variantes
12	comment eviter les blessures	QUESTION	blessure	OK - echauffement, progression graduelle
13	difference entre cardio et musculation	COMPARAISON	cardio, musculation	OK - comparaison detaillee
14	quest ce que le vo2max	DEFINITION	vo2max	OK - capacite aerobique maximale
15	comment faire des tractions	QUESTION	tractions	OK - prise, amplitude, muscles cibles
16	quest ce que le squat	DEFINITION	squat	OK - mouvement fondamental, technique
17	pourquoi faire du sport	QUESTION	sport	Partiel - reponse tres generale (entite trop large)
18	comment augmenter sa force	QUESTION	force	OK - musculation lourde, progression
19	quest ce que l'endurance	DEFINITION	endurance	OK - capacite aerobique et cardio
20	au revoir	QUITTER	-	OK - message de fin

18 questions sur 20 obtiennent une réponse correcte ou très bonne. La question 17 ("pourquoi faire du sport") produit une réponse correcte mais trop générale car l'entité "sport" est présente dans 15 paires Q/R et le TF-IDF ne peut pas distinguer la meilleure parmi des candidats également pertinents. Ce cas illustre une limite connue du système (voir section 9).

8. Metriques et evaluation

Toutes les metriques ci-dessous ont ete mesurees automatiquement via les tests et des scripts d'evaluation executes sur le systeme de production. Les resultats sont reproductibles avec la commande : `python -m pytest tests/ -v`.

8.1 Precision de la classification d'intention (20 questions)

Intention	Questions testees	Correctes	Taux
SALUTATION	"bonjour", "salut comment ca va", "hello"	3/3	100 %
QUITTER	"au revoir", "quit"	2/2	100 %
IDENTITE	"qui es-tu", "que peux-tu faire"	2/2	100 %
DEFINITION	"qu'est-ce que la musculation", "c'est quoi le cardio", "qu'est-ce que l'endurance", "definir le gainage", "qu'est-ce que la musculation"	5/5	100 %
COMPARAISON	"quelle est la difference entre BFS et DFS", "compare le football et le basketball", "difference entre running et jogging"	3/3	100 %
QUESTION	"comment faire des pompes", "combien de proteines par jour", "comment ameliorer mon endurance", "quand faire du sport", "pourquoi faire de la musculation", "quel sport pour maigrir"	5/5	100 %
TOTAL	20 questions variees	20/20	100 % (cible : ≥ 80 %)

Le resultat de 100 % decoule de la conception soigneuse des regles V1 : ordre de priorite des intentions, gestion des mots-cles multi-mots, distinction IDENTITE/QUESTION. Le Naive Bayes V2 n'est pas sollicite sur ce jeu car les regles V1 suffisent.

8.2 Precision des reponses (50 questions)

Metrique	Resultat mesure	Cible du sujet	Ecart
Precision exacte (egalite stricte)	38/50 = 76 %	≥ 70 %	+6 points
Precision approx. (≥ 3 mots communs parmi les 10 premiers)	39/50 = 78 %	≥ 70 %	+8 points

Les 11 cas de reponse incorrecte s'expliquent principalement par deux phenomenes. Premièrement, des entites trop generiques : "sprint" a peu de connexions dans le graphe et le TF-IDF classe une reponse sur l'endurance en tete. Deuxiemement, des questions de definition qui recoivent une reponse de type QUESTION sur le meme sujet : "Qu'est-ce que la musculation ?" peut recevoir "Pour un pratiquant de musculation, on recommande 1,6-2,2g..." si cette reponse partage plus de mots-cles TF-IDF avec la question.

8.3 Temps de reponse sur 20 questions

Question	Temps (ms)	Commentaire
"bonjour"	0.04	Court-circuit SALUTATION avant la recherche
"quest ce que la musculation"	2.99	Pipeline complet, 2 entites
"comment faire des pompes"	1.23	Pipeline complet, 1 entite
"combien de proteines par jour"	1.62	2 entites, pool de 6 candidats
"quel sport pour perdre du poids"	3.27	2 entites generiques, pool large
"comment ameliorer son endurance"	2.41	1 entite
"quest ce que le cardio"	1.88	1 entite
"quels exercices pour les abdominaux"	1.45	1 entite
"comment bien se recuperer apres le sport"	3.54	2 entites, pool large
"quelle alimentation pour la prise de masse"	1.73	n-gramme trigramme reconnu
"quest ce que le gainage"	0.89	1 entite tres specifique
"comment eviter les blessures au sport"	3.56	2 entites
"difference entre cardio et musculation"	4.96	COMPARAISON, 2 entites, plus de candidats
"quest ce que le vo2max"	0.86	1 entite tres specifique, reponse directe
"comment faire des tractions"	1.23	1 entite
"quest ce que le squat"	0.91	1 entite specifique
"pourquoi faire du sport"	2.79	1 entite generique, pool large
"quest ce que l'endurance"	2.13	1 entite
"comment augmenter sa force"	2.56	1 entite
"au revoir"	0.05	Court-circuit QUITTER

Question	Temps (ms)	Commentaire
MOYENNE	2.00 ms	Cible : < 2 000 ms - Facteur 1 000x sous la cible

8.4 Comparaison BFS / DFS / A* (10 requetes)

Requete	BFS expl.	DFS expl.	A* expl.	Chemin optimal
musculature -> nutrition	12	4	8	4 aretes (BFS) ou 4 aretes semantiques (A*)
football -> sport	2	3	2	1 arete directe
cardio -> vo2max	4	3	4	1 arete directe (BFS/A*) ou 2 aretes (DFS)
seche -> calories	10	5	10	3 aretes via proteines (BFS) ou via deficit (A*)
musculature -> proteines	4	2	2	1 arete directe
football -> competition	7	6	7	2 aretes via coupe_du_monde
natation -> sport	2	3	2	1 arete directe
cardio -> entrainement	3	4	3	1 arete directe
pompes -> musculature	2	2	2	1 arete directe
velo -> sport	3	3	3	1 arete directe

8.5 Impact mesure du feedback sur les poids du graphe

Cycle	Question	Score	Aretes modifiees	Poids avant -> apres
Initial	-	-	0	proteines->nutrition: 0.950, proteines->musculature: 0.800
Cycle 1	"combien de proteines par jour"	5/5	2 aretes (+0.05)	proteines->nutrition: 0.950->1.000, proteines->musculature: 0.800->0.850
Cycle 2	"combien de proteines par jour"	5/5	1 arete (+0.05)	proteines->nutrition: 1.000 (plafonne), proteines->musculature: 0.850->0.900

Cycle	Question	Score	Aretes modifiees	Poids avant -> apres
Cycle 3 (penalisation)	"quel sport pour perdre du poids"	1/5	3 aretes (-0.05)	perte_de_poids->deficit: 1.000->0.950, ->nutrition: 1.000->0.950, ->cardio: 0.900->0.850

8.6 Bilan de la suite de tests

Fichier	Tests	Couverture
test_chatbot.py	18	Pipeline complet bout en bout, relances, feedback, persistance, temps de reponse
test_knowledge_base.py	11	CRUD graphe, bornage poids, serialisation JSON, round-trip
test_nlp_engine.py	18	Pipeline NLP, stemmer, Levenshtein, extraction entites, classification intention
test_search_engine.py	16	BFS, DFS, A*, admissibilite heuristique, benchmark comparatif
test_learning_engine.py	17	TF-IDF, cosinus, Naive Bayes, feedback 3 cycles, persistance
test_ui.py	22	CLI, endpoints Flask /ask /feedback /stats, entrees invalides
TOTAL	102/102 PASSED en 0.98s	Python 3.12, Linux

9. Limites identifiées et pistes d'amélioration

9.1 Limites du systeme actuel

9.1.1 Couverture incomplete du graphe

Le graphe couvre 69 concepts, ce qui est suffisant pour les themes principaux mais laisse des lacunes pour des sujets tres courants : HIIT, yoga, running en tant que discipline, crossfit, pilates, stretching actif/passif, preparation physique professionnelle, nutrition vegetarienne ou vegan, supplements (creatine, BCAA), carences nutritionnelles. Ces absences se manifestent directement dans le test de conversation : 3 questions sur 20 (HIIT, yoga, running) recoivent le message de fallback.

9.1.2 Stemmer maison imparfait

SimpleFrenchStemmer couvre les cas courants mais echoue sur des exceptions morphologiques : "nutrition" et "nutritionnel" ne partagent pas la meme racine (le suffixe "ationnel" requiert 8 caracteres de racine avant lui). SnowballStemmer ou une lemmatisation (qui requiert un lexique) amelioreraient la couverture. Le sujet autorisait SnowballStemmer ; un refactoring l'utilisant serait simple.

9.1.3 Absence de gestion de la negation

Les stopwords incluent "ne" et "pas". La question "je ne veux pas faire de musculation" est traitee de la meme facon que "je veux faire de la musculation" : l'entite "musculation" est extraite dans les deux cas. Le chatbot peut repondre positivement a une demande negative. La correction demanderait de detecter les patterns de negation avant la suppression des stopwords et d'inverser le signal semantique de l'entite concerne.

9.1.4 Questions multi-intentions

Une question comme "Qu'est-ce que la prise de masse et comment la differencier de la seche ?" melange DEFINITION et COMPARAISON. Le systeme traite COMPARAISON (prioritaire) et neglige la composante DEFINITION. Un systeme plus avance detecterait les intentions multiples et construirait une reponse composite.

9.1.5 Re-ranking TF-IDF limite sur les questions generiques

Les questions avec des entites tres generiques ("sport", "nutrition") produisent un pool de candidats trop large pour que le TF-IDF puisse les discriminer. La reponse est souvent correcte mais trop generale. Le probleme est structurel : le graphe propose trop de candidats egalement pertinents, et le TF-IDF n'a pas les outils pour choisir le bon parmi une vingtaine de reponses semantiquement proches.

9.2 Pistes d'amélioration pour le M2

- Embeddings denses (BERT / CamemBERT) : remplacer TF-IDF par des embeddings sémantiques. CamemBERT, pré-entraîné sur du français, fournirait une représentation bien plus riche permettant de retrouver des paires Q/R pertinentes même sans correspondance de mots-clés exacts.
- Classification d'intention par deep learning : un modèle sequence-to-label (LSTM, transformer) entraîné sur quelques milliers de phrases annotées remplacerait avantageusement règles V1 + Naïve Bayes V2. Plus robuste aux formulations atypiques et aux mélanges de registres.
- Reinforcement learning pour la sélection de réponse : transformer le feedback en signal de récompense pour un agent RL (Q-learning, PPO) qui apprendrait à sélectionner la meilleure réponse adaptativement, en tenant compte du contexte conversationnel complet.
- Génération de texte : un modèle génératif (GPT, LLaMA fine-tune sur le domaine sport) pourrait générer des réponses adaptées même pour des sujets non indexés dans le graphe, éliminant les messages de fallback.
- Architecture distribuée : graphe dans une base de données graphe (Neo4j), index vectoriel (Faiss, Milvus) pour la recherche TF-IDF ou par embeddings sur des millions de concepts.
- Persistance optimisée du feedback : plutôt que de rejouer tout l'historique au démarrage, persister les poids ajustés dans un fichier séparé `knowledge_graph_appris.json` et le fusionner avec le graphe auteur au chargement. Cela éviterait un jeu de plus en plus long à mesure que l'historique grandit.

10. Utilisation de l'IA

Cette section documente précisément l'utilisation d'outils d'IA conformément aux règles du projet. L'auteur peut expliquer et défendre devant le jury chaque élément produit avec l'assistance de l'IA.

10.1 Outil utilisé

Claude (Anthropic), modèle claude-sonnet-4-6, interface web claude.ai. Aucun autre outil IA (Copilot, ChatGPT) n'a été utilisé.

10.2 Usages détaillés

10.2.1 Génération du corpus de connaissances

La création de 72 paires Q/R et de 118 relations de graphe représentait un travail de contenu important. L'IA a accéléré la phase de génération initiale.

Prompt type : "Génère 15 paires question-réponse en JSON sur la nutrition sportive, avec les champs question, answer, concepts, intent. Les réponses doivent être en français, précises, 50-150 mots. Les concepts doivent correspondre à des nœuds d'un graphe sport/nutrition. Intent = DEFINITION, QUESTION ou COMPARAISON."

Ce que l'auteur a fait ensuite : vérification de la factualité (valeurs nutritionnelles, techniques), correction des concepts inexacts ou absents du graphe, ajout des relations manquantes entre les nouveaux concepts, calibrage des poids sémantiques selon la force des relations.

10.2.2 Debug du pipeline NLP

Deux bugs subtils ont été identifiés et corrigés avec l'aide de l'IA.

Bug 1 - Double flux de tokens. La classification d'intention échouait sur les questions de définition car les tokens stemmes détruisaient les mots-clés. Le bug n'était pas visible dans les tests unitaires de NLP Engine seul. Prompt : "Voici mon ChatBot.answer() et un test qui échoue. La question 'qu'est-ce que la musculation' est classée INCONNU. Identifie le problème et propose une correction."

Bug 2 - Entités génériques écrasant les spécifiques. Le symptôme : les questions sur la nutrition retournaient toujours une réponse sur le sport en général. L'IA a aidé à identifier que combiner les scores de toutes les entités avantageait "sport" (15 paires) sur "manger" (2 paires). La solution de la recherche par entité séparée a été validée par l'auteur.

10.2.3 Generation de tests unitaires

Prompt type : "Ecris des tests pytest pour `cosine_similarity(vec_a, vec_b)`. Couvre : $\text{sim}(a,a)=1.0$, $\text{sim}(\text{orthogonal})=0.0$, $\text{sim}(\text{vide},b)=0.0$, commutativite, cas avec valeurs reelles TF-IDF typiques."

Ce que l'auteur a fait : verification mathematique des valeurs attendues (calcul a la main), ajout de cas specifiques au corpus du projet, execution des tests pour valider leur correction.

10.4 Taux d'utilisation estime

Environ 35 a 40 % du code de production a ete initie ou debugge avec l'assistance de l'IA, puis systematiquement relu, modifie et valide par l'auteur. Les algorithmes centraux (Levenshtein, TF-IDF, Naive Bayes, BFS, DFS, A*) ont ete etudies en cours et leurs implementations ont ete verifiees formellement. La conception architecturale (double flux, recherche par entite, persistance) resulte entierement de decisions prises par l'auteur face aux problemes rencontres en developpement.

11. Bibliographie et references

11.1 Ouvrages de reference

- Russell, S. & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4e edition). Pearson. Chapitres 3 (BFS, DFS, A*, heuristiques admissibles), 12 (inference probabiliste, Naive Bayes), 23 (NLP, tokenisation, classification de texte). Reference principale pour les algorithmes et leur analyse de complexite.
- Manning, C. D., Raghavan, P. & Schutze, H. (2008). Introduction to Information Retrieval. Cambridge University Press. Disponible en ligne : <https://nlp.stanford.edu/IR-book/>. Chapitres 6 (TF-IDF), 13 (Naive Bayes pour la categorisation de texte), 14 (similarite vectorielle, cosinus). Reference principale pour le module LearningEngine.
- Jurafsky, D. & Martin, J. H. (2023). Speech and Language Processing (3e edition, draft). <https://web.stanford.edu/~jurafsky/slp3/>. Chapitres 2 (tokenisation, stemming, normalisation), 4 (Naive Bayes et classification), 8 (etiquetage de sequences, NER). Reference principale pour le module NLPEngine.

11.2 Documentation technique

- Documentation Python 3.12 - modules utilises : heapq (tas binaire A*), collections.deque (file BFS), collections.defaultdict (compteurs TF-IDF), json, re (expressions regulieres), math. <https://docs.python.org/3.12/>
- Flask documentation v3.0 - application web, routage, gestion JSON, CORS. <https://flask.palletsprojects.com/en/3.0.x/>
- pytest documentation v8.0 - tests unitaires, fixtures, assertions, parametrisation. <https://docs.pytest.org/en/stable/>

11.3 Cours et materiaux pedagogiques

- Cours d'Intelligence Artificielle M1 TRIED - Adrien ESCOURROU - IPST-CNAM Albi, 2025-2026. Contenu sur BFS, DFS, A* et heuristiques, TF-IDF, Naive Bayes, feedback loop.
- Cours Traitement du Signal et des Donnees M1 TRIED - Concepts de representation vectorielle et de similarite.

11.4 IA utilisee

- Claude (Anthropic) - Modele claude-sonnet-4-6 - <https://claude.ai> - Utilise pour la generation de paires Q/R, le debug, la generation de tests, et la redaction de ce rapport. Usage detaille documente en section 10.